

Resposta questão A:

A mudança para o **desenvolvimento assistido por IA** não elimina a necessidade do engenheiro júnior — ela muda o “centro de gravidade” da profissão. O foco sai da escrita manual de código e vai para a **capacidade de pensar, estruturar e validar soluções**. Quem apenas “digita código” tende a perder espaço; quem entende o problema profundamente passa a ter mais valor.

Duas competências se tornam indispensáveis:

1. Pensamento lógico e modelagem de problemas (competência técnica)

O desenvolvedor precisa saber decompor um problema em etapas claras, definir regras, fluxos e restrições antes mesmo de gerar código. Ferramentas como Gemini conseguem implementar rapidamente, mas dependem de instruções bem estruturadas.

Em vez de perguntar “como codar isso?”, o júnior precisa saber responder “qual é exatamente o problema e como ele deve funcionar?”.

2. Engenharia de prompt e comunicação técnica (competência híbrida)

Saber “conversar com a IA” vira uma habilidade central. Isso inclui escrever instruções claras, iterar respostas, validar saídas e refinar resultados. Não é só dar comandos — é saber **explicar contexto, restrições e objetivos** de forma precisa.

Quem domina isso consegue extrair soluções melhores, mais rápidas e com menos erros.

Além dessas, há um complemento importante:

3. Pensamento crítico e validação (comportamental/técnica)

A IA erra, simplifica ou inventa soluções. O profissional precisa revisar código, identificar inconsistências e garantir qualidade.

Não basta gerar código — é preciso saber se ele está correto, seguro e eficiente.

Impacto geral na formação:

O engenheiro júnior deixa de ser um “executor de código” e passa a ser um **orquestrador de soluções**, alguém que entende o problema, guia a IA e valida o resultado. Quem não desenvolver essas competências tende a se tornar dependente da ferramenta — e facilmente substituível.

Resposta questão B:

2. Diretriz ética para equilibrar inovação e proteção

Uma solução eficaz não é proibir, mas **estruturar o uso responsável**. Aqui vai uma proposta prática:

Princípio da “Transformação Significativa + Transparência”

a) Transformação Significativa (técnica)

Toda solução gerada a partir de referência deve apresentar mudanças claras em pelo menos um destes níveis:

- **Estrutura** (arquitetura, organização do código)
- **Experiência** (UX/UI, fluxos)
- **Propósito** (novo contexto ou problema resolvido)

Se nada disso muda, não é inovação — é cópia.

b) Transparência (ética/profissional)

- Documentar quando houve inspiração externa.
- Em ambientes corporativos, manter registros de uso de IA no desenvolvimento.
- Em projetos públicos, citar referências quando apropriado (como já ocorre no open source).

c) Auditoria assistida por IA (organizacional)

Empresas podem usar IA não só para gerar, mas para **detectar similaridade excessiva** entre soluções — criando uma espécie de “antiplágio de software”.

Conclusão

O avanço de ferramentas generativas não elimina a ética — ele **exige mais dela**. O diferencial do desenvolvedor não será quem copia mais rápido, mas quem **transforma melhor**.

Se a IA vira um atalho para reproduzir, temos estagnação.

Se vira um meio para reinterpretar e evoluir ideias, temos inovação real.

Relatório de melhorias implementadas:

Envio de imagem, podendo alterar tamanho e posição.

Remover fundo com recurso I.A

Adicionar texto, podendo alterar tamanho cor e posição.

Adicionado tamanhos e formatos para utilização em diferentes plataformas

<https://github.com/diegobmferrari/AmebaMaker.git>

<https://ai.studio/apps/fb3c9a9a-48ec-4134-8d5b-c4edde7c9b97>